

Scanning Networks

Host Discovery, Port Scanning & Service/OS Enumeration

Study Notes — Lab Walkthrough on the CEH Practice Network
(Target range: 10.10.1.0/24, domain CEH.com)

■■ *These scans are demonstrated only inside an isolated CEH lab range. Never scan a network or host you do not own or have explicit written authorization to test — unauthorized scanning may be illegal.*

What is Scanning Networks?

Scanning is the **second phase** of ethical hacking, right after footprinting. Once a target's IP range/domain is known, scanning is used to find out **which hosts are alive**, **which ports are open**, and **what services/OS** are running on them. This builds a live map of the network that is used to plan exploitation.

Three main goals of scanning

- **Host Discovery** — is the machine online? (ping sweep / ARP scan)
- **Port Scanning** — which TCP/UDP ports are open, closed, or filtered?
- **Service & OS Fingerprinting** — what software/version and operating system is running on those open ports?

■ **Key takeaway:** Scanning turns a single IP address or subnet into a detailed inventory of live hosts, open doors (ports), and the exact software behind each door — the foundation for vulnerability analysis.

1. Host Discovery — nmap Ping Scans

Tool: Nmap (-sn = no port scan, just check if host is alive)

Before scanning ports, it's efficient to first confirm the target is actually online. Nmap offers multiple probe types for this.

```
root@parrot:~# nmap -sn -PR 10.10.1.22
Nmap scan report for 10.10.1.22
Host is up (0.00038s latency).
MAC Address: 00:15:5D:01:80:02 (Microsoft)
Nmap done: 1 IP address (1 host up) scanned in 0.22 seconds

root@parrot:~# nmap -sn -PU 10.10.1.22
Nmap scan report for 10.10.1.22
Host is up (0.00059s latency).
MAC Address: 00:15:5D:01:80:02 (Microsoft)
Nmap done: 1 IP address (1 host up) scanned in 0.16 seconds
```

- **-sn** → "ping scan": only checks if the host is up, skips port scanning entirely
- **-PR** → ARP Ping — sends an ARP request; works only on the local network (Layer 2), extremely fast and reliable
- **-PU** → UDP Ping — sends a UDP packet to a (usually closed) port and waits for an ICMP port-unreachable reply to confirm the host is alive
- The **MAC Address** vendor lookup (00:15:5D... = Microsoft) hints this is a Hyper-V virtual machine

■ **Key takeaway:** ARP ping (-PR) is the fastest and most reliable host-discovery method on a local subnet because ARP cannot be blocked by a host-based firewall the way ICMP/TCP probes can.

2. TCP Connect Scan — Zenmap (nmap -sT)

Tool: Zenmap (GUI front-end for Nmap) — Windows

Scan type: -sT (TCP Connect Scan) -v (verbose)

A TCP Connect scan completes the **full 3-way TCP handshake** (SYN → SYN/ACK → ACK) with every port. It's the most reliable scan type but also the easiest for the target to log, since a full connection is established.

Command: `nmap -sT -v 10.10.1.22`

Port	State	Service
53/tcp	open	domain
80/tcp	open	http
88/tcp	open	kerberos-sec
135/tcp	open	msrpc
139/tcp	open	netbios-ssn
389/tcp	open	ldap
445/tcp	open	microsoft-ds
464/tcp	open	kpasswd5
593/tcp	open	http-rpc-epmap
636/tcp	open	ldaps
1433/tcp	open	ms-sql-s
3268/tcp	open	globalcatLDAP
3269/tcp	open	globalcatLDAPssl
3389/tcp	open	ms-wbt-server
5985/tcp	open	wsman
50006/tcp	open	unknown

- 980 of 1000 scanned ports were **filtered** (no response) — a firewall is silently dropping probes
- Ports 88 (Kerberos), 389/636 (LDAP), 3268/3269 (Global Catalog) → this is a **Windows Active Directory Domain Controller**
- Port 445 (microsoft-ds/SMB), 1433 (MS-SQL), 3389 (RDP), 5985 (WinRM) are all classic AD-environment services

■ **Key takeaway:** Seeing ports 88, 389, 445, 3268 together is a strong fingerprint of a Windows Domain Controller — recognizing these port combinations by sight is a core CEH skill.

3. SYN Stealth Scan — Zenmap (nmap -sS)

Scan type: -sS (TCP SYN / "half-open" scan) -v (verbose)

The SYN scan sends only the first SYN packet and reads the reply, then sends a RST instead of completing the handshake — hence "half-open" or "stealth" scan. It's faster than -sT and less likely to be logged by the target application (though modern firewalls/IDS still detect it).

```
nmap -sS -v 10.10.1.22
Completed SYN Stealth Scan at 01:53, 4.52s elapsed (1000 total ports)
Nmap scan report for 10.10.1.22
Not shown: 981 filtered tcp ports (no-response)
Raw packets sent: 1983 (87.236KB) | Rcvd: 21 (908B)
```

- Same open ports discovered as the TCP Connect scan (53, 80, 88, 135... 3389, 5985, 50006)
- **Only 4.52 seconds** vs 5.18s for the full connect scan — SYN scan is faster since it never finishes the 3-way handshake
- "Raw packets sent: 1983" shows Nmap directly crafts packets at a low level rather than using the OS socket API (requires root/admin privileges)

■ **Key takeaway:** -sT vs -sS: Connect scan = full handshake, reliable but noisy/loggable. SYN scan = half-open, faster and stealthier, but needs raw-socket (root/admin) privileges.

4. OS & Service Fingerprinting — nmap -A

Scan type: -A (Aggressive: OS detection + version detection + script scan + traceroute)

The aggressive scan combines multiple techniques into one command, giving the most complete picture of a single host in one pass.

```
smb2-security-mode:
3:1:1:
Message signing enabled and required
smb-security-mode:
account_used: guest
authentication_level: user
challenge_response: supported
message_signing: required
smb-os-discovery:
OS: Windows Server 2022 Standard 20348 (Windows Server 2022 Standard 6.3)
Computer name: Server2022
NetBIOS computer name: SERVER2022
Domain name: CEH.com
Forest name: CEH.com
FQDN: Server2022.CEH.com
System time: 2026-07-04T02:16:00-07:00
nbstat: NetBIOS name: SERVER2022, NetBIOS user: <unknown>
smb2-time:
date: 2026-07-04T09:16:00

TRACEROUTE
HOP RTT ADDRESS
1 0.62 ms 10.10.1.22
```

- **smb-os-discovery script** reveals: exact OS = Windows Server 2022 Standard, computer name Server2022, and that it belongs to the **CEH.com** Active Directory domain/forest
- **smb2-security-mode** shows SMB message signing is **enabled and required** — this actually blocks some relay-style attacks (e.g. NTLM relay) against this host
- **smb-security-mode** shows a **guest** account is used for the anonymous session, at **user-level authentication**
- The **system time** retrieved via SMB can reveal timezone / patch-cycle clues without any login
- **TRACEROUTE** is bundled into the same aggressive scan — hop 1 is the target itself, meaning it's on the same local segment as the scanner

■ **Key takeaway:** SMB enumeration through Nmap scripts (smb-os-discovery, smb2-security-mode) can fingerprint the exact OS build, computer/domain name, and SMB signing policy — all without valid credentials.

5. Packet-Level Analysis — Wireshark

Tool: Wireshark (packet capture & analysis)

While Nmap tells you the scan result, Wireshark shows the actual packets flowing on the wire — useful for verifying what a scan really does at the protocol level, or for general network traffic analysis during/after scanning.

Notable frames observed in the capture:

- **mDNS (Multicast DNS)** traffic — repeated "Standard query response TXT, cache flush" packets from IPv6 link-local addresses (fe80::...) — devices announcing themselves on the local network (common with printers, IoT, Apple/Bonjour devices)
- **ICMPv6 Router Solicitation** — a host asking local routers to identify themselves (IPv6 auto-configuration)
- **TCP RST, ACK** highlighted packet: source **20.190.148.166:443** → destination **10.10.1.11:54539** — a remote HTTPS connection being **reset/torn down**
- **DNS queries** immediately after: 10.10.1.11 asks 8.8.8.8 for an **A record for licensing.mp.microsoft.com** — routine Windows license/activation check-in traffic

■ **Key takeaway:** Wireshark lets you verify scan behaviour and spot background "noise" traffic (mDNS, licensing check-ins, IPv6 auto-config) that could otherwise be mistaken for suspicious activity.

6. Scanning from Metasploit — nmap Integration

Tool: Metasploit Framework (msfconsole) — built-in nmap command

Metasploit lets you launch Nmap directly from msfconsole and automatically import the results into the current workspace database for later exploitation.

```
msf6 >> nmap -Pn -sS -A -oX Test 10.10.1.0/24
[*] exec: nmap -Pn -sS -A -oX Test 10.10.1.0/24

Nmap scan report for 10.10.1.2
Host is up (0.00066s latency).
Not shown: 998 filtered tcp ports (no-response)
PORT STATE SERVICE VERSION
53/tcp open  domain Unbound
88/tcp open  http  nginx
|_http-title: pfSense - Login
MAC Address: 02:15:5D:21:A2:2A (Unknown)
```

- **-Pn** → skip host discovery, treat all hosts as online (useful when ICMP/ping is blocked)
- **-sS -A** → SYN stealth scan + aggressive OS/service/script detection, same as earlier sections
- **-oX Test** → save results in **XML format** to a file named 'Test' (Metasploit can auto-import this into its database of hosts/services)
- Result: 10.10.1.2 is running **pfSense** (an open-source firewall/router distribution) with an nginx-based web login portal on port 88 and Unbound DNS on port 53

■ **Key takeaway:** Running Nmap inside Metasploit means every discovered host, port, and service is automatically stored in the Metasploit database — ready to feed straight into exploit modules.

7. Metasploit Auxiliary Module — SYN Port Scanner

Module: auxiliary/scanner/portscan/syn

Metasploit also ships its own native port-scanning modules (not relying on the external Nmap binary), configured the same way as any other auxiliary/exploit module: with set options and run.

```
msf6 >> use auxiliary/scanner/portscan/syn
msf6 auxiliary(scanner/portscan/syn) >> set INTERFACE eth0
INTERFACE => eth0
msf6 auxiliary(scanner/portscan/syn) >> set PORTS 80
PORTS => 80
msf6 auxiliary(scanner/portscan/syn) >> set RHOSTS 10.10.1.5-23
RHOSTS => 10.10.1.5-23
msf6 auxiliary(scanner/portscan/syn) >> set THREADS 50
THREADS => 50
msf6 auxiliary(scanner/portscan/syn) >> run

[+] TCP OPEN 10.10.1.9:80
[+] TCP OPEN 10.10.1.19:80
[+] TCP OPEN 10.10.1.22:80
[*] Scanned 19 of 19 hosts (100% complete)
[*] Auxiliary module execution completed
```

- **INTERFACE** — which local network interface to scan from (eth0)
- **PORTS** — restrict the scan to just port 80 (HTTP) across a whole IP range, for speed
- **RHOSTS** — target range 10.10.1.5 through 10.10.1.23 (19 hosts)
- **THREADS** — 50 parallel scanning threads for faster results across many hosts
- Result: 3 hosts out of 19 respond with port 80 open (10.10.1.9, 10.10.1.19, 10.10.1.22) — these are the web servers on the subnet

■ **Key takeaway:** Metasploit's own scanner modules are handy when you want to stay inside msfconsole for the entire workflow — set a few options, run, and go straight into exploitation without switching tools.

Quick-Revision Summary Table

Command / Module	Purpose	Notes
<code>nmap -sn -PR</code>	ARP host discovery	Fastest, local network only (Layer 2)
<code>nmap -sn -PU</code>	UDP host discovery	Uses ICMP port-unreachable to confirm alive host
<code>nmap -sT</code>	TCP Connect scan	Full 3-way handshake, reliable, more visible/loggable
<code>nmap -sS</code>	SYN stealth scan	Half-open scan, faster, needs raw-socket privilege
<code>nmap -A</code>	OS/service/script/traceroute	One command = full host fingerprint (incl. SMB info)
Wireshark	Packet-level verification	See actual handshakes, RSTs, DNS/mDNS background traffic
<code>msf nmap -oX</code>	Nmap from Metasploit	-oX saves XML for auto-import into MSF database
<code>auxiliary/scanner/portscan/syn</code>	Native MSF port scan	set RHOSTS/PORTS/THREADS then run

Exam / Interview Tips

- Know the difference: **-sT** (full connect, reliable, noisy) vs **-sS** (half-open, stealthier, faster).
- Recognize AD/Domain Controller signatures: ports 88 (Kerberos), 389/636 (LDAP), 3268/3269 (Global Catalog).
- **-Pn** skips host discovery — essential when a firewall blocks ping but ports are still reachable.
- SMB scripts (`smb-os-discovery`, `smb2-security-mode`) can reveal OS build and domain/forest name pre-auth.
- Metasploit can both call external Nmap *and* run its own auxiliary scanner modules — know both paths.
- Filtered ports = firewall silently drops probes; Closed ports = host responds RST (port not listening).